

Rapport – Projet Intelligence Artificielle

Jeu de dames

Membres de groupe

- Adel Nekiche
- Marouane messafri

Année universitaire 2025-2026

Sommaire:

1. Introduction.....	3
2. Présentation du jeu.....	3
2.1 Description générale.....	3
2.2 Règles principales.....	4
3. Architecture du projet.....	4
3.1 Organisation des dossiers.....	4
4. Implémentation du moteur de jeu 4.....	5
4.1 Représentation du plateau.....	5
4.2 Gestion des déplacements.....	6
4.3 Détection de fin de partie.....	6
4.4 Compatibilité et interface graphique:.....	6
5. Intelligence Artificielle.....	8
5.1 Niveau Facile – Minimax.....	8
5.2 Niveau Moyen – Alpha-Beta.....	8
5.3 Niveau Difficile – IA Avancée.....	8
6. Fonctions d'évaluation.....	9
7. Algorithmes utilisés.....	10
7.1 Minimax.....	10
7.2 Alpha-Beta.....	10
8. Tests et validation.....	10
9. Analyse expérimentale.....	11
9.1 Comparaison des fonctions d'évaluation:.....	11
9.2 Tournois entre IA.....	12
10. Utilisation de l'IA générative.....	14
11. Difficultés rencontrées.....	14
12. Conclusion.....	15

1. Introduction

Dans le cadre de l'UE Intelligence Artificielle, nous avons réalisé un projet consistant à développer un jeu de dames intégrant plusieurs intelligences artificielles de différents niveaux de difficulté.

L'objectif principal de ce projet est de concevoir un moteur de jeu fonctionnel, d'implémenter plusieurs stratégies d'intelligence artificielle basées sur les algorithmes Minimax et Alpha-Beta, puis d'évaluer expérimentalement leurs performances.

Le projet a été développé en Python avec une architecture modulaire permettant une bonne séparation entre la logique métier, les algorithmes d'IA et l'interface graphique.

2. Présentation du jeu

2.1 Description générale:

Le jeu choisi est *le jeu de dames*.

Il s'agit d'un jeu stratégique à deux joueurs se jouant sur un plateau de 8×8 cases. Les pièces occupent uniquement les cases jouables (noires). Chaque joueur débute avec 12 pions disposés sur les trois premières rangées de son camp.

Le but du jeu est de capturer toutes les pièces adverses ou de bloquer totalement l'adversaire afin qu'il ne puisse plus effectuer de déplacement.

2.2 Règles principales:

Les règles implémentées dans notre projet sont les suivantes :

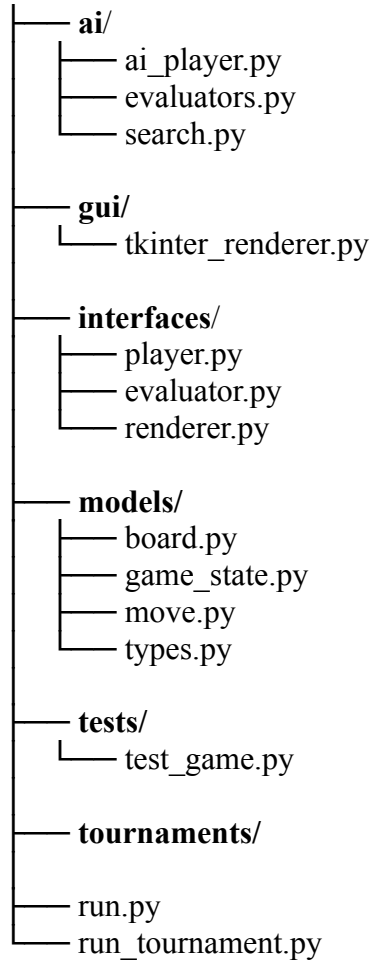
1. **Déplacement:** les pions se déplacent d'une case en diagonale vers l'avant (selon leur couleur). Les dames (rois) se déplacent en diagonale et peuvent parcourir plusieurs cases (mouvement long, type « flying king »).
2. **Capture:** la capture est obligatoire si au moins une prise est possible. Les prises multiples (séquences de sauts) sont gérées récursivement et retournées comme coups distincts.
3. **Promotion:** un pion atteint la dernière rangée adverse et est automatiquement promu en dame.
4. **Conditions de fin:** une partie se termine lorsqu'un joueur n'a plus de pièces actives ou n'a aucun coup légal. Dans ces cas, l'adversaire est déclaré gagnant.
5. **Nulle:** une partie est nulle si 20 coups consécutifs sont joués sans capture, ou si les deux joueurs répètent la même position 5 fois.

3. Architecture du projet

3.1 Organisation des dossiers:

Le projet suit une architecture propre et modulaire.

Jeu-Dames-AI/



Description des dossiers

Dossier	Rôle
models	Contient les classes représentant le plateau, les coups et l'état du jeu
ai	Contient les algorithmes d'intelligence artificielle
gui	Interface graphique Tkinter
interfaces	Interfaces abstraites du projet
tests	Tests unitaires
tournaments	Gestion des tournois automatiques

4. Implémentation du moteur de jeu

4.1 Représentation du plateau:

Le plateau est représenté sous forme de matrice 8×8.

Les valeurs utilisées sont :

Valeur	Signification
0	Case vide
1	Pion blanc
3	Pion noir
2	Dame blanche
4	Dame noire

4.2 Gestion des déplacements:

Pour chaque pièce, le moteur :

1. **La représentation d'un coup** est assurée par la classe **Move**. Elle stocke un path, c'est-à-dire la liste des positions parcourues par la pièce, ainsi qu'un ensemble **captured_positions** contenant les pièces capturées. La propriété **is_capture** permet d'identifier si le coup est une prise.
2. **La génération des coups** est centralisée dans `GameState.generate_legal_moves()`. Cette méthode commence par rechercher toutes les captures possibles grâce à `_generate_capture_moves()` et `_find_capture_sequences()`. Si au moins une capture existe, seules les captures sont retournées, ce qui permet de respecter la règle de capture obligatoire.
3. Si aucune capture n'est possible, le moteur génère les déplacements simples avec `_generate_simple_moves()`. Les pions avancent d'une case en diagonale selon leur couleur, tandis que les dames peuvent parcourir plusieurs cases en diagonale tant que les cases sont libres.
4. **L'application d'un coup** est réalisée par `Board.apply_move(move)`. Cette méthode met à jour la grille, déplace la pièce, supprime les pièces capturées, effectue la promotion automatique si un pion atteint la dernière ligne, puis change le joueur courant.

4.3 Détection de fin de partie:

Une partie se termine lorsqu'un joueur ne possède plus de pièces ou lorsqu'il ne peut plus effectuer de coup légal.

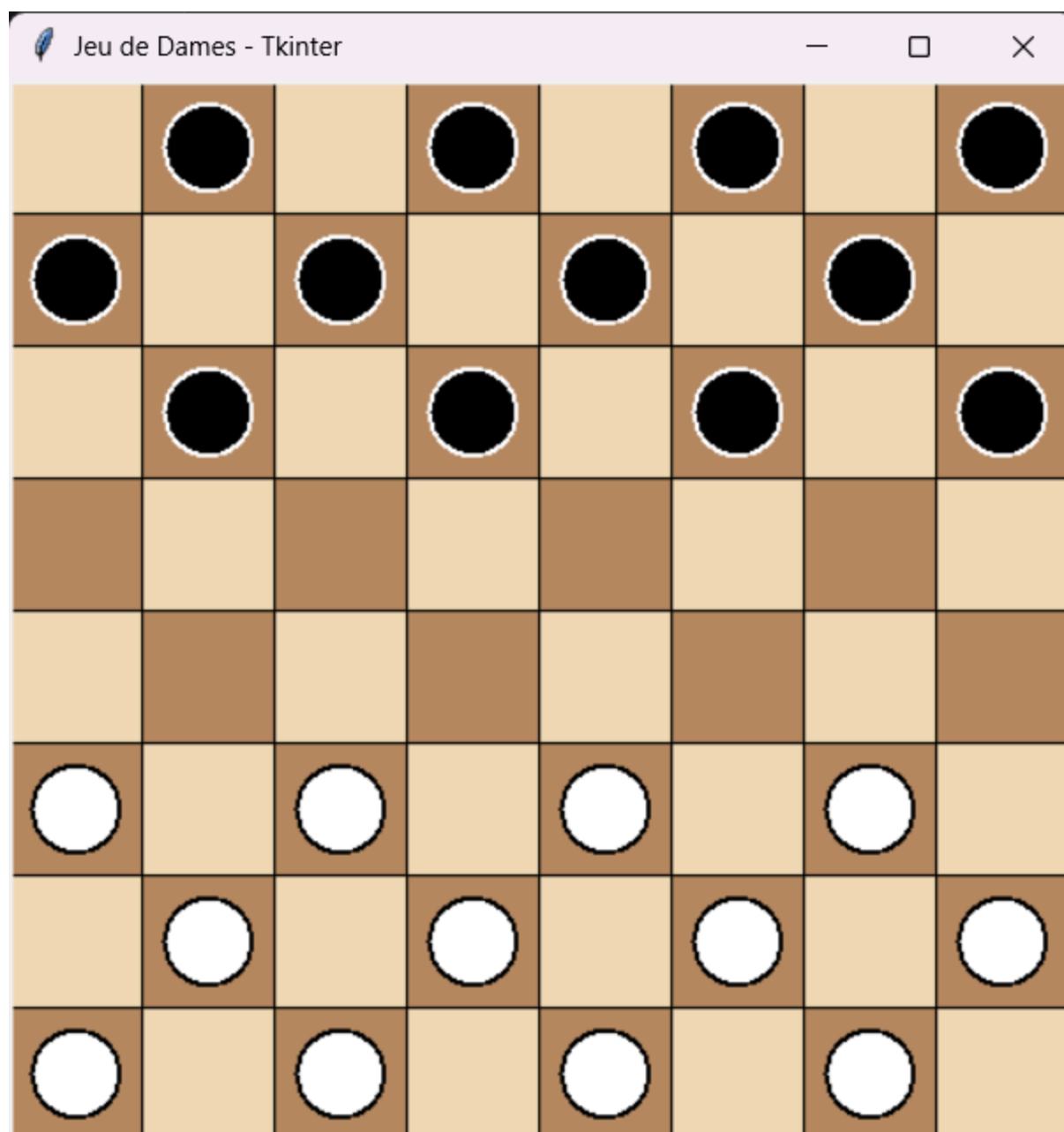
- **La détection des états terminaux** est assurée par les méthodes `GameState.is_game_over()` et `GameState.get_winner()`. Le moteur vérifie l'absence de coups légaux afin de déterminer si la partie est terminée et d'identifier le gagnant.
- Aussi, il y a également une règle de match nul basée sur un compteur de coups sans capture ainsi qu'un système de détection de répétition de positions. Si plus de 20 coups sont joués sans aucune capture, ou si la même position est répétée 5 fois, la partie est automatiquement déclarée nulle.

4.4 Compatibilité et interface graphique:

Le projet inclut **une interface graphique** développée avec **Tkinter**. Cette interface permet d'afficher le plateau de jeu, de sélectionner les pièces à la souris, d'exécuter les coups légaux et d'afficher les messages de fin de partie.

Le choix de **Tkinter** permet également d'assurer une **compatibilité multiplateforme**. Le projet peut être exécuté **sous Windows et Linux avec Python 3.11+**.

Sous Windows, Tkinter est généralement inclus avec Python. Sous Linux, il peut être nécessaire d'installer le module `python3-tk` avant l'exécution du projet.



5. Intelligence Artificielle

Le projet implémente trois niveaux d'intelligence artificielle. Chaque niveau utilise une profondeur de recherche et une fonction d'évaluation différentes afin d'adapter la difficulté.

5.1 Niveau Facile – Minimax:

Le premier niveau utilise l'algorithme Minimax avec une profondeur faible.

Caractéristiques

- algorithme : Minimax
- profondeur : 1
- évaluateur : Material
- stratégie simple
- faible temps de calcul.

Cette IA explore les coups possibles jusqu'à une profondeur limitée, puis choisit le coup ayant le meilleur score.

5.2 Niveau Moyen – Alpha-Beta:

Le niveau moyen utilise l'algorithme Alpha-Beta, une optimisation de Minimax qui permet de réduire le nombre de nœuds explorés.

Caractéristiques

- algorithme : Alpha-Beta
- profondeur : 3
- évaluateur : Material+Mobility
- meilleure qualité de jeu
- temps de calcul raisonnable

L'évaluation prend en compte le matériel ainsi que la mobilité, c'est-à-dire le nombre de coups disponibles pour chaque joueur.

5.3 Niveau Difficile – IA Avancée:

Le niveau difficile utilise également Alpha-Beta, mais avec une profondeur plus élevée et une fonction d'évaluation plus complète.

Caractéristiques

- algorithme : Alpha-Beta ;
- profondeur : 5 ;
- évaluateur : Advanced ;
- heuristique stratégique plus fine..

Critères utilisés

- valeur des pièces
- mobilité
- contrôle du centre
- menace de promotion
- défense de la dernière rangée.

Cette IA produit le meilleur niveau de jeu du projet, au prix d'un temps de calcul plus élevé.

6. Fonctions d'évaluation

Trois fonctions d'évaluation sont implémentées. Elles retournent un score indiquant si une position est favorable ou défavorable au joueur courant.

- **score positif** : position favorable au joueur courant ;
- **score négatif** : position favorable à l'adversaire ;
- **score proche de 0** : position équilibrée.

Évaluateur	Critères utilisés	Calcul du score	Interprétation
Material	Valeur des pièces	somme(joueur courant) - somme(adversaire), avec pion = 1 et dame = 5	Avantage matériel pur
Material+Mobility	Matériel + mobilité (le nombre de coups possibles)	score matériel + 0.1 × (coups du joueur courant - coups de l'adversaire)	Avantage matériel et activité
Advanced	Matériel + mobilité + contrôle du centre + protection de la dernière rangée + proximité de promotion	Material+Mobility + bonus positionnels	Évaluation stratégique plus fine

7. Algorithmes utilisés

7.1 Minimax:

À chaque feuille de l'arbre de recherche, c'est-à-dire lorsque la profondeur maximale est atteinte ou lorsqu'un état terminal est rencontré, la fonction d'évaluation calcule un score.

L'algorithme Minimax propage ensuite ce score dans l'arbre :

- Le joueur MAX cherche à maximiser le score ;
- Le joueur MIN cherche à minimiser le score.

7.2 Alpha-Beta:

Alpha-Beta applique la même logique que Minimax, mais utilise deux bornes :

- alpha : meilleur score déjà trouvé pour MAX
- beta : meilleur score déjà trouvé pour MIN.

Lorsqu'une branche ne peut plus améliorer le résultat final, elle est ignorée. Cela permet de réduire le nombre de nœuds explorés et donc d'augmenter la profondeur de recherche.

Aussi on utilise également un tri des coups avant l'exploration :

- captures en premier
- promotions ensuite
- autres coups après.

Ce tri améliore l'efficacité de l'élagage Alpha-Beta, car les bons coups sont explorés plus tôt.

8. Tests et validation

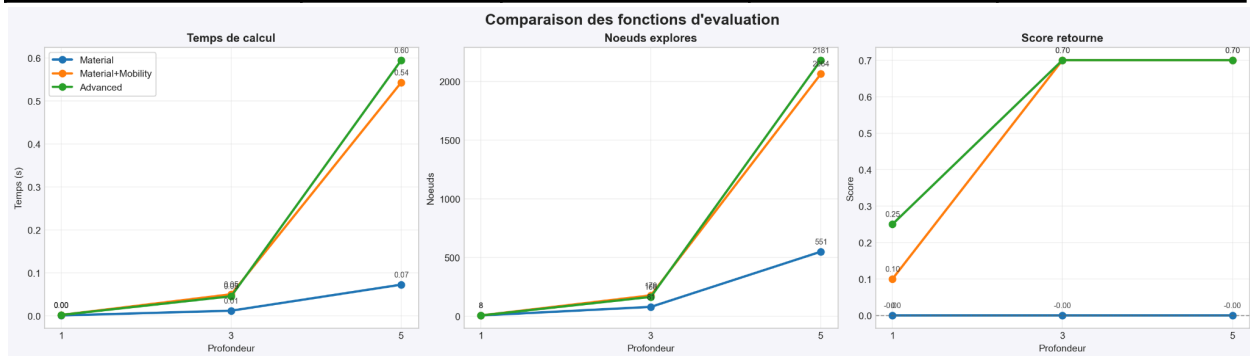
Le projet inclut une suite de tests unitaires dans `tests/test_game.py` afin de vérifier les aspects critiques du moteur de jeu. Les tests couvrent notamment l'initialisation du plateau, la génération des coups légaux, la règle de capture obligatoire, la promotion des pièces, les transitions d'état du jeu, les fonctions d'évaluation ainsi que les algorithmes de recherche Minimax et Alpha-Beta.

La suite de tests contient actuellement 14 tests unitaires, tous validés avec succès sur l'environnement de développement.

9. Analyse expérimentale

9.1 Comparaison des fonctions d'évaluation:

Évaluateur	Profondeur	Score	Nœuds explorés	Temps (s)
Material	1	0	8	0.0013
Material	3	0	81	0.124
Material	5	0	551	0.730
Material+Mobility	1	0.1	8	0.0024
Material+Mobility	3	0.7	179	0.0506
Material+Mobility	5	0.7	2064	0.5434
Advanced	1	0.25	8	0.0026
Advanced	3	0.7	166	0.0460
Advanced	5	0.7	2181	0.5952



Les résultats montrent que plus la profondeur augmente, plus le nombre de nœuds explorés et le temps de calcul augmentent. L'évaluateur Material est le plus rapide, tandis que Advanced est plus coûteux mais fournit une analyse plus stratégique.

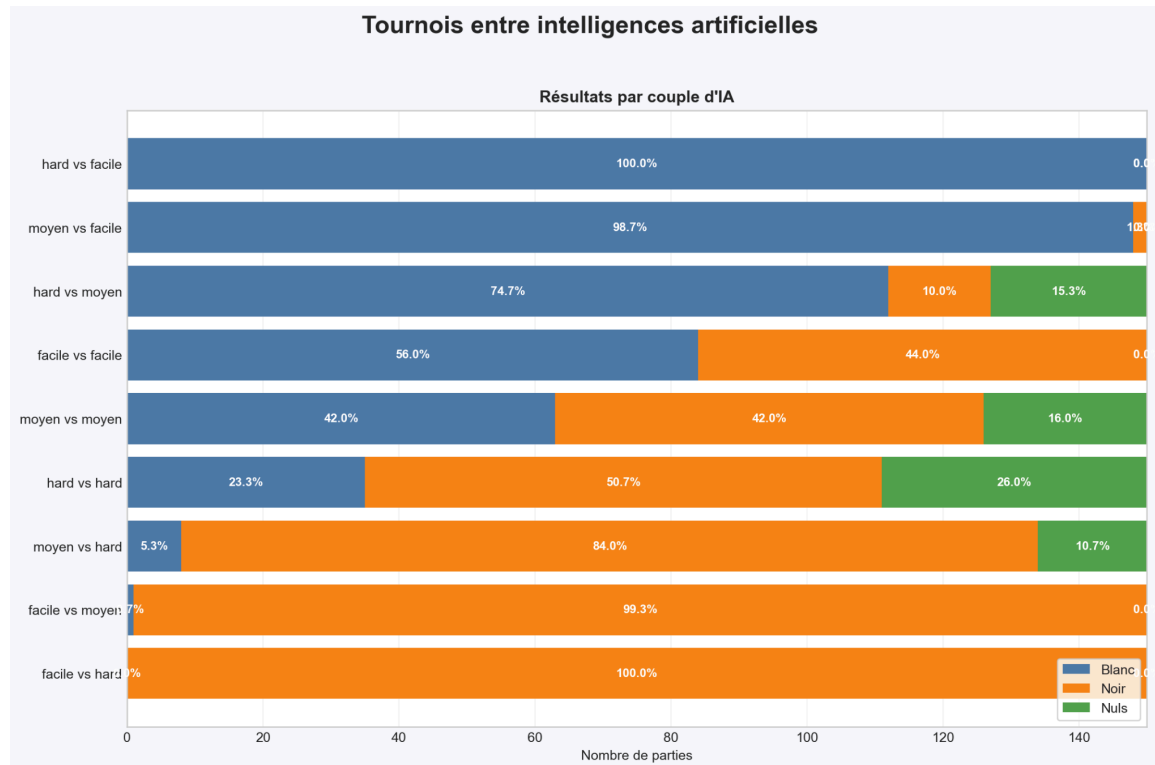
9.2 Tournois entre IA:

Afin d'évaluer concrètement les performances des différents niveaux d'intelligence artificielle, un système de tournoi automatisé a été mis en place.

Chaque confrontation entre deux niveaux d'IA a été exécutée sous forme de 3 lots de 50 parties, soit un total de 150 parties par couple d'IA. Cette approche permet d'obtenir des statistiques plus fiables et de réduire l'influence d'une série atypique de parties.

BLANC	NOIR	LOTS	Victoires blanches moy.	Victoires noires moy.	Nuls moy.	Temps moyen (s)
Facile	Facile	3	28	22	0	0.44
Facile	Difficile	3	0	50	0	35.99
Facile	moyen	3	0.33	49.67	0	2.55
Difficile	Facile	3	50	0	0	26.33
Difficile	Difficile	3	11.67	25.33	13	725.87
Difficile	moyen	3	37.33	5	7.67	261.78
moyen	Facile	3	49.33	0.67	0	2.19
moyen	Difficile	3	2.67	42	5.33	159.43
moyen	moyen	3	21	21	8	9.41

Le graphique suivant résume visuellement les résultats des tournois :



Analyse des résultats :

Les résultats montrent que :

- Les confrontations montrent clairement une hiérarchie entre les niveaux d'IA. Les IA de niveau supérieur dominent systématiquement les niveaux inférieurs. L'IA difficile gagne toutes ses parties contre l'IA facile et domine également l'IA moyenne.
- Les confrontations entre IA de même niveau produisent davantage de parties nulles et des résultats plus équilibrés. Cela est particulièrement visible pour les confrontations moyen vs moyen et hard vs hard, où les deux IA possèdent des capacités similaires.
- Le moteur introduit une légère variabilité grâce au **choix aléatoire** parmi plusieurs meilleurs coups de même score. Cette variabilité permet d'éviter des parties totalement identiques et influence légèrement les statistiques finales, surtout dans les confrontations équilibrées.
- **L'avantage du premier joueur** existe dans certaines situations, mais il reste moins important que la qualité de l'algorithme utilisé. Par exemple, l'IA moyenne domine largement l'IA facile même lorsque cette dernière joue en premier.
- Les temps moyens augmentent fortement avec la profondeur de recherche. Les confrontations impliquant l'IA difficile sont beaucoup plus coûteuses en temps de calcul, car elles utilisent une profondeur plus élevée ainsi qu'une heuristique avancée.

Ces résultats confirment la cohérence des choix algorithmiques du projet. L'augmentation de la profondeur de recherche, l'utilisation d'Alpha-Beta et les fonctions d'évaluation avancées améliorent significativement le niveau de jeu de l'intelligence artificielle.

10. Utilisation de l'IA générative

Durant le projet, plusieurs outils d'intelligence artificielle générative ont été utilisés, notamment ChatGPT et GitHub Copilot.

Ces outils ont été utilisés comme assistance au développement pour :

- comprendre certains concepts
- corriger des erreurs
- améliorer la documentation technique
- proposer des idées d'optimisation
- assister la réalisation des benchmarks et des scripts d'analyse
- aider à la création et à l'amélioration de certaines parties de l'interface graphique Tkinter.

Les outils d'IA n'ont cependant pas remplacé le travail de conception et de développement. Le code proposé a systématiquement été relu, testé, modifié et intégré manuellement afin de garantir sa cohérence avec l'architecture du projet et les besoins fonctionnels.

11. Difficultés rencontrées

Les principales difficultés rencontrées durant le développement du projet concernent principalement l'implémentation correcte des règles du jeu ainsi que l'optimisation des performances de l'intelligence artificielle.

La gestion des captures multiples a représenté une difficulté importante, car il fallait permettre à une même pièce d'effectuer plusieurs prises successives tout en respectant les règles officielles du jeu de dames. La règle de capture obligatoire a également nécessité une logique spécifique lors de la génération des coups légaux.

Le choix et la conception des heuristiques d'évaluation ont également demandé plusieurs expérimentations afin d'obtenir un comportement cohérent de l'intelligence artificielle. Différents critères ont été ajoutés progressivement, comme la mobilité, le contrôle du centre, la menace de promotion.

Les temps d'exécution des tournois ont également représenté une contrainte importante. Certaines confrontations impliquant l'IA difficile nécessitent plusieurs minutes pour terminer 50 parties, en raison de la profondeur de recherche élevée et des heuristiques avancées utilisées. Cela a nécessité plusieurs optimisations afin de rendre les simulations de tournois exploitables dans un temps raisonnable.

Pour résoudre ces problèmes, plusieurs améliorations ont été mises en place :

- optimisation de certaines fonctions critiques
- tri des coups avant exploration
- ajout de tests unitaires afin de valider le comportement du moteur de jeu.
- diminution de la profondeur de l'IA difficile de 6 à 5 afin de réduire le temps d'exécution des tournois.

12. Conclusion

Ce projet nous a permis d'appliquer concrètement plusieurs notions importantes liées à l'intelligence artificielle, aux algorithmes de recherche et au développement logiciel.

Nous avons développé un moteur de jeu de dames complet intégrant les règles principales du jeu, une interface graphique ainsi que plusieurs niveaux d'intelligence artificielle basés sur Minimax et Alpha-Beta.

Le projet nous a également permis :

- d'implémenter différentes heuristiques d'évaluation ;
- de comparer les performances de plusieurs niveaux d'IA ;
- d'analyser l'impact de la profondeur de recherche ;
- d'étudier l'efficacité de l'élagage Alpha-Beta ;
- de réaliser des benchmarks et des tournois automatisés entre IA.

Les résultats obtenus montrent clairement que l'utilisation d'Alpha-Bêta, du tri des coups et d'heuristiques plus avancées améliore significativement la qualité de jeu de l'intelligence artificielle, tout en réduisant le nombre de nœuds explorés.

Enfin, ce projet nous a permis de renforcer nos compétences en programmation Python, en architecture logicielle modulaire, en conception d'algorithmes et en intelligence artificielle appliquée aux jeux.